PROJECT PROPOSAL

ATLAS AQAR

Agentic AI–Driven Real-Estate Research and Investment Intelligence System

Four-Week Integrated Development Proposal

Academic project proposal prepared as a unified team effort

Project: Atlas AQAR

Objective

Atlas AQAR aims to develop a bounded, multi-agent artificial intelligence system that supports real-estate research and investment analysis through autonomous task decomposition, evidence-grounded retrieval, property search, financial computation, iterative verification, and transparent report generation. The system extends the Atlas agentic research architecture into a real-estate domain while integrating the requirements of a real-estate search and investment advisory assistant. The objective is to reduce the time and effort required to compare properties, payment plans, market information, and approximate investment outcomes within a single coherent workflow.

Description

Atlas AQAR is designed as an end-to-end agentic research platform rather than a conventional sequential chatbot. Given a natural-language investment or property-search request, the system interprets the user's constraints, decomposes the request into dependency-aware research tasks, selects appropriate tools, gathers and validates evidence, performs deterministic financial calculations, synthesizes findings, and critiques its own draft before producing the final response.

The architecture preserves Atlas's core mechanisms: a Planner for structured decomposition, a ReAct-based Researcher for information foraging, a Writer for evidence-grounded synthesis, and a Critic for verification and conditional re-routing. LangGraph provides the state-machine orchestration layer, while Pydantic-based contracts maintain typed shared state and explicit evidence binding.

The real-estate domain layer introduces property and developer search, installment-plan analysis, approximate ROI estimation, scenario analysis, comparison workflows, real-estate RAG, metadata filtering, session preferences, caching, and a user-facing dashboard. The system is explicitly designed to distinguish verified evidence from assumptions and to terminate gracefully when sufficient verifiable information is unavailable.

Tools & Design

- Agentic orchestration through a cyclic LangGraph workflow with conditional routing rather than a fixed sequential pipeline.
- Planner that converts a compound request into approximately three to six prioritized and dependency-mapped research tasks.
- ReAct-based Researcher that dynamically selects retrieval, web, property, developer, or calculation tools according to the current state.
- Evidence Arsenal containing structured evidence identifiers, task associations, source metadata, retrieval method, and supporting text.
- Writer that produces citation-grounded reports in which material claims are linked to explicit evidence identifiers.
- Critic that checks evidence coverage, citation validity, numerical consistency, constraint satisfaction, unsupported claims, and unresolved conflicts, with targeted re-routing after rejection.
- Real-estate tools for property search, property details, developer lookup, installment calculation, ROI estimation, and scenario analysis.
- Retrieval-first RAG with controlled web fallback when local evidence is insufficient.
- Bounded autonomy through retry caps, graph-iteration limits, token/cost budgets, timeouts, validation, and graceful insufficient-evidence exits.
- Session context and preference memory for relevant user constraints, together with caching for repeated or slow-changing requests.

- Interactive dashboard exposing research progress, property comparisons, financial results, evidence, and final reports.
- Integrated evaluation through automated tests, retrieval evaluation, ablation studies, trajectory analysis, adversarial testing, and performance monitoring.

Technologies to Use

Layer	Technology	Purpose
Programming	Python	Core implementation, deterministic tools, testing and orchestration
Agent Orchestration	LangGraph	Stateful, cyclic multi-agent workflow and conditional routing
LLM / Structured Output	LLM API + Pydantic	Reasoning, planning, structured contracts and validation
RAG / Vector Store	Chroma + embeddings	Semantic retrieval over real-estate documents
Web Research	Controlled web-search API	Current external information when local evidence is insufficient
Backend	FastAPI	Versioned service layer and API endpoints
Frontend	Streamlit	Interactive research and comparison dashboard
Observability	Langfuse or LangSmith	Tracing, token/cost monitoring and trajectory analysis
Evaluation	RAGAS + custom suite	Groundedness, retrieval and end-to-end performance evaluation
Testing / Deployment	Pytest + Docker	Automated verification and reproducible deployment

Team Structure & Unified Responsibilities

Atlas AQAR will be developed as a unified team effort. Responsibilities are organized by technical workstreams rather than by named individuals or individually assigned proposal sections. All workstreams share the same architecture, repository, evaluation criteria, integration process, and release objectives.

Integrated Workstream	Collective Responsibility
Agentic Intelligence & Orchestration	Design and implement the Planner, Researcher, Writer, Critic, LangGraph state, routing logic, bounded autonomy and agentic control flow.
Data, RAG & Evidence	Prepare the property/document corpus, implement ingestion and retrieval, manage metadata, maintain evidence provenance, and improve retrieval quality.
Property & Financial Intelligence	Implement property/developer tools, installment calculations, ROI estimation, scenario analysis, validation and financial consistency checks.
Backend & System Integration	Develop the API, persistence, caching, configuration, error handling, service interfaces and integration contracts.
Interface & Analytics	Develop the Streamlit dashboard, research-progress visualization, property comparison, financial presentation, evidence inspection and usage analytics.
Quality, Evaluation & Deployment	Maintain automated testing, benchmark datasets, observability, evaluation metrics, adversarial testing, Dockerization, deployment and release validation.

Performance Matrix

The matrix below preserves the evaluation logic of the Atlas architecture and refines it for the real-estate application. Results should be measured at the end of implementation and reported with the final system demonstration.

Criterion	Target / Threshold	Measurement Approach	Evidence of Achievement
Task Completion	≥ 80% on the defined end-to-end benchmark	Automated execution across answerable, complex, unanswerable and contradictory cases	Benchmark report and representative traces
Citation Precision	> 90%	Programmatic verification that cited evidence identifiers exist and support associated claims; human spot checks	Citation-validation results
Groundedness / Faithfulness	High and improved over non-RAG baseline	RAGAS-style evaluation plus manual review of sampled reports	Evaluation dashboard and sample reports

Criterion	Target / Threshold	Measurement Approach	Evidence of Achievement
Financial Accuracy	100% on deterministic calculation tests	Comparison against independently calculated reference values	Unit and integration test results
Evidence Coverage	High coverage of material claims	Supported important claims ÷ total important claims	Evidence-coverage report
Critic Efficacy	Measurable improvement from initial to final draft	Compare Draft 1 and final output for factual, citation and completeness corrections	Critic trajectory analysis
Unsupported-Claim Rate	Near-zero for verified report sections	Automated and human audit of claims lacking adequate evidence	Audit report
Latency & Cost	Measured and bounded within project limits	Trace latency, token usage and estimated cost; compare cached and uncached requests	Observability dashboard
Robustness	No unhandled failures in scripted adversarial suite	Prompt injection, missing data, conflicting sources, malformed inputs and edge-case tests	QA test report

Week 1: Foundation, Data & Single-Agent Core

Tasks and responsibilities are expressed at the integrated team level to emphasize coordinated delivery across the project workstreams.

Tasks

- Freeze the technical architecture, typed state contracts, repository structure, data schemas, evaluation protocol, and integration interfaces.
- Prepare a representative real-estate dataset of approximately 30–50 property listings together with supporting developer, location, payment-plan, and market documents, preserving source metadata and dates.
- Implement the RAG ingestion pipeline, including cleaning, chunking, metadata assignment, embeddings, Chroma indexing, retrieval, and initial retrieval tests.
- Implement deterministic property, developer, installment, ROI, and scenario-analysis tools with strict schemas and unit tests.
- Build the standalone Atlas Researcher ReAct loop and connect it to local retrieval, controlled web fallback, and the real-estate tool registry.
- Establish the initial Streamlit prototype, API skeleton, automated testing structure, configuration management, logging, and Docker foundation.

Deliverables

- Validated real-estate dataset and document corpus.
- Working Chroma-based RAG pipeline with metadata filtering.
- Validated property and financial tool suite.
- Standalone Researcher capable of evidence-grounded real-estate research.
- Initial API and dashboard prototypes.
- Gate 1: representative questions answered with explicit sources and reproducible tool traces.

Week 2: Multi-Agent Orchestration & Integrated Intelligence

Tasks and responsibilities are expressed at the integrated team level to emphasize coordinated delivery across the project workstreams.

Tasks

- Implement the complete LangGraph state schema and conditional workflow.
- Implement the Planner with structured research plans, priorities, dependencies, and completion criteria.
- Extend the Researcher into a domain-aware research agent capable of dynamically selecting retrieval, web, property, developer, and financial tools.
- Implement the Writer and Evidence Arsenal so material report claims are explicitly linked to source evidence.
- Implement the Critic with checks for evidence coverage, citation validity, financial consistency, user constraints, missing information, and unresolved conflicts.
- Implement conditional re-routing, retry caps, failure handling, untrusted-data isolation, token/cost budgets, and end-to-end integration tests.

Deliverables

- Functional Planner → Research → Analysis → Writer → Critic workflow.
- Typed shared state and structured evidence records.
- Conditional research-again loop with bounded autonomy.
- Citation-grounded investment research report.
- Validated financial-analysis workflow.
- Gate 2: the full cycle successfully completes the defined majority of benchmark questions and recovers from targeted critique failures.

Week 3: Evaluation, Observability, Performance & Hardening

Tasks and responsibilities are expressed at the integrated team level to emphasize coordinated delivery across the project workstreams.

Tasks

- Integrate Langfuse or LangSmith tracing and expose agent trajectories, tool calls, latency, token usage, cost estimates, and retry behavior.
- Build and execute a 25–50 case evaluation suite covering normal, complex, unanswerable, contradictory, adversarial, and financial scenarios.
- Measure retrieval quality, groundedness, citation precision, evidence coverage, unsupported claims, financial accuracy, task completion, latency, cost, and error rates.
- Conduct the comparative ablation study: Base LLM vs. LLM + RAG vs. LLM + RAG + Tools vs. full Atlas AQAR.
- Harden the system against prompt injection in retrieved content, conflicting sources, missing data, malformed listings, impossible constraints, calculation edge cases, and excessive agent loops.
- Refine the dashboard, API, caching strategy, error handling, evaluation reports, and system documentation based on measured failures.

Deliverables

- Live observability/tracing dashboard.
- Benchmark dataset and automated evaluation pipeline.
- Performance matrix with measured results.
- Ablation study and comparative analysis.
- Adversarial and robustness test report.
- Gate 3: feature freeze with documented limitations and validated system behavior.

Week 4: Final Integration, Deployment & Demonstration

Tasks and responsibilities are expressed at the integrated team level to emphasize coordinated delivery across the project workstreams.

Tasks

- Freeze the architecture and stabilize all agents, tools, APIs, retrieval parameters, prompts, guardrails, and user-facing workflows.
- Complete the final Streamlit dashboard with research progress, property comparison, financial analysis, evidence inspection, and final report views.
- Finalize the versioned FastAPI service, caching, configuration, Docker deployment, logging, and production-style error handling.
- Perform final end-to-end validation against the performance matrix and resolve all critical defects.
- Prepare the final technical documentation, architecture diagrams, limitations, evaluation results, ablation charts, and reproducible demonstration workflow.
- Prepare cached backup runs and a deterministic live-demo path to reduce presentation risk while preserving a fully deployable system.

Deliverables

- Deployed Atlas AQAR system.
- Final Streamlit dashboard and versioned API.
- Completed performance matrix and evaluation report.
- Final architecture, technical, and limitations documentation.
- Reproducible live demonstration and backup execution traces.
- Final deliverable package ready for academic evaluation and presentation.

Output Structure

The final Atlas AQAR output follows a structured, evidence-grounded format so that the system's research products remain transparent and auditable. The output structure retains the Atlas evidence-and-verification principle while refining it for real-estate research and investment analysis.

Output Section	Purpose	Required Content
Executive Summary	Provide a concise overview without obscuring uncertainty.	User objective, key findings, principal constraints, major assumptions, and evidence status.
Research Scope & Criteria	Make research boundaries explicit.	Budget, location, property type, bedrooms, delivery horizon, investment objective, and stated constraints.
Property Findings	Present relevant properties and verified attributes.	Project/property name, location, developer, price, area, bedrooms, delivery, payment plan and evidence.
Comparative Analysis	Enable transparent comparison.	Side-by-side property attributes, payment-plan structure, financial assumptions, and market context.
Financial Analysis	Present deterministic calculations separately from qualitative reasoning.	Down payment, installment schedule, total payment, ROI estimate, scenarios, assumptions and calculation provenance.
Market & Contextual Findings	Connect individual properties to broader evidence.	Location characteristics, developer information, market signals, infrastructure or contextual factors supported by sources.

Output Section	Purpose	Required Content
Evidence & Citations	Make factual foundations auditable.	Evidence IDs, source references, retrieval method, dates where available, and claim-to-evidence relationships.
Uncertainty & Limitations	Prevent false precision.	Missing data, conflicting sources, assumptions, confidence limitations, and unresolved questions.
Final Synthesis	Integrate evidence into a coherent research conclusion.	Concise evidence-grounded synthesis aligned with the user's stated criteria, without fabricating missing information.

Architecture and Quality Principles

- Evidence before assertion: material claims must be grounded in retrieved or verified evidence whenever such evidence is required.
- Deterministic computation: arithmetic and financial calculations are performed by validated tools rather than generated as free-form language.
- Bounded autonomy: the system stops when evidence is sufficient, budgets are exhausted, or the configured iteration limit is reached.
- Transparent uncertainty: insufficient or conflicting information is reported explicitly rather than concealed through confident language.
- Untrusted-data isolation: retrieved web content is treated as data and never as executable instructions.
- Separation of concerns: agentic reasoning determines what to investigate, while deterministic services perform calculations, storage, filtering, validation, and infrastructure operations.
- Reproducibility: evaluation cases, tool outputs, evidence records, configuration, and deployment procedures should be reproducible.
- Academic measurability: every major architectural claim is paired with a measurable evaluation method or demonstrable artifact.

Expected Final Outcome

At completion, Atlas AQAR will constitute a unified agentic AI system for real-estate research and investment intelligence. The final system will combine autonomous research planning, ReAct-based tool use, real-estate RAG, property and developer search, financial analysis, evidence management, iterative critique, bounded autonomy, observability, an interactive dashboard, and a versioned API. The project will be assessed not only by functional demonstration but also through quantitative performance measurement, ablation analysis, robustness testing, and transparent documentation of limitations.